



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



4. Historia de XML

- XML significa lenguaje de marcado extensible
- XML fue diseñado para almacenar y transportar datos
- XML fue diseñado para ser legible tanto por humanos como por máquinas
- XML es una recomendación del W3C
- XML no hace nada

Ejemplo:

```
<nota>  
  <para>Mis alumnos</para>  
  <de>Profesora Patricia</de>  
  <título>Recordatorio</título>  
  <mensaje>No olviden estudiar para su próximo examen</mensaje>  
</nota>
```

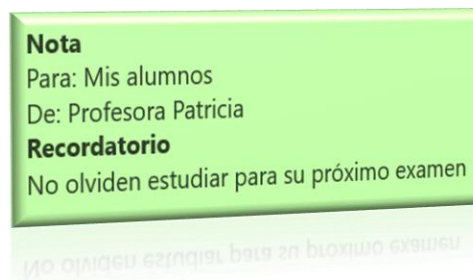
El ejemplo es muy descriptivo:

Tiene información del remitente y del receptor,
Tiene un título, y
Tiene un mensaje

} Pero no hace nada

Es solo información envuelta en etiquetas.

Alguien deberá escribir un software para enviarlo, recibirlo, almacenarlo o mostrarlo.



XML y HTML fueron diseñados con diferentes objetivos:

- XML fue diseñado para transportar datos, enfocado en qué datos son
- HTML fue diseñado para mostrar datos, enfocado a cómo se ven los datos
- Las etiquetas XML no están predefinidas como las etiquetas HTML
- Las etiquetas en XML son inventadas por el autor del documento XML.
- Con XML, el autor deberá definir la estructura del documento y las etiquetas.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



XML es Extensible

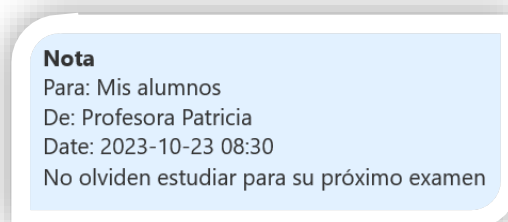
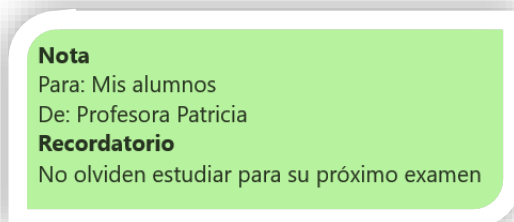
La mayoría de las aplicaciones XML funcionarían como se espera incluso si se agregan (o eliminan) nuevos datos.

Imagine una aplicación diseñada para mostrar la versión original de note.xml (`<para><de><titulo><mensaje>`).

Luego imagine una versión más nueva de note.xml con elementos `<date>` y `<hour>` agregados, y un `<titulo>` eliminado.

La forma XML es construida, la versión anterior de la aplicación aún puede funcionar:

```
<nota>
  <date>2020-03-13</date>
  <hour>12:30</hour>
  <para>Mis alumnos</para>
  <de>Profesora Patricia</de>
  <mensaje>No olviden estudiar para su próximo examen</mensaje>
</nota>
```



XML simplifica las cosas

- Simplifica el intercambio de datos
- Simplifica el transporte de datos
- Simplifica los cambios de plataforma
- Simplifica la disponibilidad de datos

Muchos sistemas informáticos contienen datos en formatos incompatibles. El intercambio de datos entre sistemas incompatibles (o sistemas actualizados) es una tarea que requiere mucho tiempo para los desarrolladores web. Se deben convertir grandes cantidades de datos y, a menudo, se pierden datos incompatibles.

XML almacena datos en formato de texto plano. Esto proporciona una forma independiente de software y hardware de almacenar, transportar y compartir datos.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



XML también facilita la expansión o actualización a nuevos sistemas operativos, nuevas aplicaciones o nuevos navegadores, sin perder datos.

Con XML, los datos pueden estar disponibles para todo tipo de "máquinas de lectura" como personas, computadoras, máquinas de voz, noticias, etc.

XML es una recomendación de la W3C desde febrero de 1998.

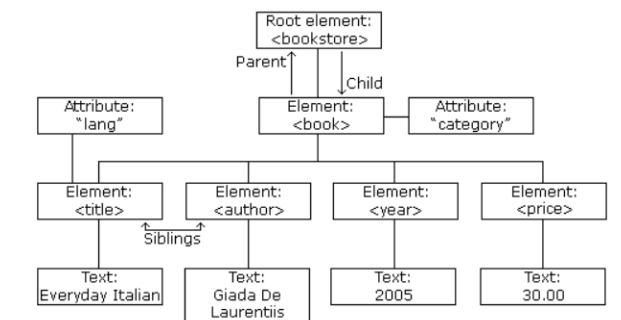
- XML se usa a menudo para separar los datos de la presentación.
- XML no contiene ninguna información sobre cómo mostrarse.
- Los mismo datos XML se pueden usar en muchos escenarios de presentación diferentes, debido a esto, con XML existe la separación total entre los datos y la presentación.
- XML se usa para almacenar o transportar datos, mientras que HTML se usa para formatear y mostrar los mismos datos.
- Al mostrar datos en HTML, no deberíamos tener que editar el archivo HTML cuando los datos cambian.
- Con pocas líneas de código JavaScript, puede leer un archivo XML y actualizar el contenido de datos de cualquier página HTML.

Existen miles de formatos XML, en industrias diferentes, para describir las transacciones diarias de datos:

- Transacciones financieras
- Datos Médicos
- Datos matemáticos
- Mediciones científicas
- Información de noticias
- Servicios meteorológicos

4.2 Elementos de XML

Los documentos XML forman una estructura de árbol que comienza en "la raíz" y se ramifica a "las hojas".





U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Los documentos XML son formados como árboles de elementos.

Un árbol XML inicia con el elemento root y sus ramas son elementos child.

Todos los elementos pueden tener sub elementos.

```
<root>  
  <child>  
    <subchild>.....</subchild>  
  </child>  
</root>
```

Los términos parent, child y sibling son usados para describir las relaciones entre los elementos.

Sintaxis

XML utiliza una sintaxis muy autodescriptiva.

```
<?xml versión="1.0" encoding="UTF-8"?>
```

El prólogo es opcional, pero si se pone debe de ser al inicio del documento.

Ejemplo de la librería:

- La primera línea es el elemento root del documento: <libreria>
- La siguiente línea comienza con un elemento: <book>
- El elemento <book> tiene 4 elementos: <title>, <autor>, <year> y <price>
- Y después se coloca la línea final del elemento </book>

Un **elemento** puede contener:

- Texto
- Atributos
- Otros elementos
- O una combinación

Elementos vacíos: <element></element> ó <element />

Reglas de nomenclatura XML

Los elementos XML deben seguir estas reglas de nomenclatura:

- Los nombres de los elementos distinguen entre mayúsculas y minúsculas
- Los nombres de los elementos deben comenzar con una letra o un guión bajo



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



- Los nombres de elementos no pueden comenzar con las letras xml (o XML, o Xml, etc.)
- Los nombres de elementos pueden contener letras, dígitos, guiones, guiones bajos y puntos.
- Los nombres de elementos no pueden contener espacios
- **Se puede usar cualquier nombre, no se reservan palabras (excepto xml)**

Mejores prácticas de nomenclatura

- Cree nombres descriptivos como: <person>, <firstname>, <lastname>.
- Cree nombres simples y cortos como: <book_title> no <the_title_of_the_book>.
- **Evite “-”**. Por ejemplo: si nombra algo como “first-name”, algunos programas pueden pensar que desea obtener “name” de “first”.
- **Evite “.”**. Por ejemplo: si nombra algo como “first.name”, algunos programas pueden pensar que “name” es una propiedad del objeto “first”.
- **Evite “:”**. Ya que están reservados para espacios de nombres.

Estilos de nombres

No hay un estilo de nombres definido para elementos XML, pero los siguientes son los usados comúnmente:

Style	Example	Description
Lower case	<firstname>	All letters lower case
Upper case	<FIRSTNAME>	All letters upper case
Underscore	<first_name>	Underscore separates words
Pascal case	<FirstName>	Uppercase first letter in each word
Camel case	<firstName>	Uppercase first letter in each word except the first

Los documentos XML tienen frecuentemente una correspondiente base de datos. Una práctica común es usar las reglas de nomenclatura de la base de datos para los elementos XML.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Como se mencionó anteriormente todos los elementos XML pueden tener atributos, como en HTML. Éstos son diseñados para contener información relacionada a un específico elemento. Los atributos siempre deben estar entre comillas, por ejemplo:

```
<person gender="female">
```

Si el valor del atributo contiene comillas, puedes usar comillas simples, como esto:

```
<gangster name='George "Shotgun" Ziegler'>
```

Ó utilizar los caracteres: `<gangster name="George "Shotgun" Ziegler">`

Los atributos también pueden ser representados como elementos, no hay reglas acerca de cuándo usar atributos o cuando elementos en XML.

Atributos XML para metadatos

Algunas veces las referencias ID son asignadas a los elementos, es decir para identificar elementos XML de la misma forma que el atributo id en HTML. En el siguiente código, podemos observar que el atributo ID, nos permite identificar diferentes notas:

```
<messages>
  <note id="501">
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
  </note>
  <note id="502">
    <to>Jani</to>
    <from>Tove</from>
    <heading>Re: Reminder</heading>
    <body>I will not</body>
  </note>
</messages>
```

Es decir, los metadatos (datos sobre datos) deben almacenarse como atributos, y los datos en sí como elementos.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Conflicto de nombres

Debido a que los nombres de los elementos son definidos por los desarrolladores, cuando mezclamos varios documentos XML de diferentes aplicaciones XML, frecuentemente se genera un conflicto, por lo que, para resolver esto debemos utilizar un prefijo de nombre:

```
<h:table>
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table>
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

Para usar prefijos en XML deben ser definidos por un **namespace**. El namespace puede ser definido como un atributo **xmlns** en la etiqueta de inicio de un elemento, teniendo la siguiente sintaxis:

xmlns:prefijo="URI" (por ejemplo: **xmlns:h="http://www.w3.org/TR/html4/"**) o en el elemento root del XML: **<root xmlns:h=http://www.w3.org/TR/html4/>**

El propósito de usar un URI es para dar un único nombre al namespace, no para buscar información.

Un **Identificador Uniforme de Recursos (URI)** es una cadena de caracteres que identifica un recurso de Internet.

El URI más común es el Localizador Uniforme de Recursos (URL) que identifica una dirección de dominio de Internet.



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Marcado Extensible (XML)



4.3 Definición del tipo de documento (DTD)

Un DTD define la estructura y los elementos y atributos legales de un documento XML.

En un DTD, los elementos son declarados con la siguiente sintaxis:

`<!ELEMENT element-name category>`

or

`<!ELEMENT element-name (element-content)>`

Un documento XML “valido” esta “bien formado”, si cumple con las reglas de un DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

La declaración DOCTYPE contiene una referencia a un archivo DTD. El contenido del DTD, es el siguiente:

Note.dtd:

```
<!DOCTYPE note
[
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
```

Éste es interpretado así:

- ✓ **!DOCTYPE note** – Define que el elemento root del documento es note



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



- ✓ **!ELEMENT note** – Define que el elemento note, debe contener los elementos: “to, from, heading y body”
- ✓ **!ELEMENT to** – Define que el elemento to, sea de tipo “#PCDATA”
- ✓ **!ELEMENT from** – Define que el elemento from, sea de tipo “#PCDATA”
- ✓ **!ELEMENT heading** – Define que el elemento heading, sea de tipo “#PCDATA”
- ✓ **!ELEMENT body** – Define que el elemento body, sea de tipo “#PCDATA”

El tipo **#PCDATA**, es un dato caracter que se puede analizar. Como, por ejemplo, el texto que se encuentra entre la etiqueta de inicio y la etiqueta final de un elemento XML. Un analizador examinará ese texto en busca de entidades y marcas. Las etiquetas dentro del texto se tratarán como marcas y las entidades se expandirán.

CDATA significa datos de caracteres. Es texto que no será analizado. Las etiquetas dentro del texto NO serán tratadas como marcas y las entidades no serán expandidas.

También podemos declarar un DTD dentro de un archivo XML, éste deberá estar dentro de la definición `<!DOCTYPE>`, de la siguiente forma:

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</body>
</note>
```



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Uso de DTD para declaración de entidad

Una declaración DOCTYPE puede ser usada para definir caracteres especiales o cadenas, usadas en el documento y ésta tiene 3 partes: comienza con un **&**, luego viene el **nombre de la entidad** y termina con un **;**.

Lo anterior lo podemos visualizar en el siguiente ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE note [
  <!ENTITY nbsp "&#xA0;">
  <!ENTITY writer "Writer: Donald Duck.">
  <!ENTITY copyright "Copyright: W3Schools.">
]>

<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
  <footer>&writer;&nbsp;&copyright;</footer>
</note>
```

```
▼ <note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
  <footer>Writer: Donald Duck. Copyright: W3Schools.</footer>
</note>
```

Con un DTD puedes verificar que los datos que recibes del mundo exterior sean válidos, también puedes usar un DTD para verificar tus propios datos, se puede acordar entre un grupo de personas usar un DTD estándar para intercambiar datos.

XML no requiere un DTD, cuando trabaje con archivos XML pequeños, crear una DTD puede ser una pérdida de tiempo.

Si se desarrollan aplicaciones, espera hasta que la especificación sea estable antes de agregar un DTD, sino el software podría dejar de funcionar debido a errores de validación.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



4.4 Esquema de documentos XML (XML Schema)

Un esquema XML describe la estructura de un documento XML, justo como un DTD.

El lenguaje de esquema XML también se conoce como definición de esquema XML (XSD).

El Esquema XML es una alternativa basada en XML a DTD:

```
<xs:element name="note">

  <xs:complexType>
    <xs:sequence>
      <xs:element name="to" type="xs:string"/>
      <xs:element name="from" type="xs:string"/>
      <xs:element name="heading" type="xs:string"/>
      <xs:element name="body" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

</xs:element>
```

Y el cual es interpretado así:

- ✓ **<xs:element name="note">** - Define el elemento llamado “note”
- ✓ **<xs:complexType>** - El elemento “note” es de tipo complejo
- ✓ **<xs:sequence>** - El tipo complejo es una secuencia de elementos
- ✓ **<xs:element name="to" type="xs:string">** - El elemento “to” es de tipo cadena (texto)
- ✓ **<xs:element name="from" type="xs:string">** - El elemento “from” es de tipo cadena
- ✓ **<xs:element name="heading" type="xs:string">** - El elemento “heading” es de tipo cadena
- ✓ **<xs:element name="body" type="xs:string">** - El elemento “body” es de tipo cadena

El propósito de un esquema XML es definir los bloques de construcción legales de un documento XML:

- ✓ Los elementos y atributos pueden aparecer en un documento
- ✓ El número de (y el orden de) elementos child
- ✓ Tipos de datos para elementos y atributos
- ✓ Valores predeterminados y fijos para elementos y atributos



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



En el mundo XML, cientos de formatos XML estandarizados son diariamente usados. Muchos de esos estándares están definidos por esquemas XML.

Los esquemas XML son más potentes que los DTD:

- ✓ **Son escritos en XML:**
 - No tienes que aprender un nuevo lenguaje
 - Puedes utilizar su editor XML para editar sus archivos esquema
 - Puedes usar su analizador para analizar sus archivos esquema
 - Puedes manipular tus esquemas con XML DOM
 - Puedes transformar tus esquemas con XSLT
- ✓ **Son extensibles a las adiciones**, porque están escritos en XML:
 - Reutilice su esquema en otros esquemas
 - Cree sus propios tipos de datos derivados de los tipos estándar
 - Referencie múltiples esquemas en el mismo documento
- ✓ **Soportan tipos de datos** (Una de las mayores fortalezas de los esquemas XML es la compatibilidad con los tipos de datos):
 - Es más fácil describir el contenido del documento permitido
 - Es más fácil definir restricciones en los datos
 - Es más fácil validar la exactitud de los datos
 - Es más fácil definir patrones de datos
 - Es más fácil convertir datos entre diferentes tipos de datos
- ✓ **Soportan namespaces**

¿Por qué usar un esquema XML?

Con esquema XML, los archivos XML pueden llevar una descripción de su propio formato, se puede acordar un estándar para intercambiar datos y se pueden verificar los datos.

Comunicación Segura de Datos

Cuando se envían datos de un remitente a un receptor, es esencial que ambas partes tengan las mismas "expectativas" sobre el contenido.

Con los esquemas XML, el remitente puede describir los datos de manera que el receptor los entienda.

Una fecha como: "03-11-2004", en algunos países, se interpretará como 3. Noviembre y en otros países como 11. Marzo.

Sin embargo, un elemento XML con un tipo de datos como este: **<date type = "date"> 2004-03-11 </date>** garantiza una comprensión mutua del contenido, porque el tipo de datos XML "fecha" requiere el formato "AAAA-MM-DD".



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Para referenciar un esquema en un documento XML, se debe hacer de la siguiente forma:

```
<?xml version="1.0"?>

<note xmlns="https://www.w3schools.com"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://www.w3schools.com note.xsd">

  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

`xmlns="https://www.w3schools.com"` especifica la declaración de espacio de nombres predeterminada. Esta declaración le dice al validador de esquema que todos los elementos utilizados en este documento XML se declaran en el espacio de nombres "https://www.w3schools.com".

Una vez que tenga disponible el espacio de nombres de instancia de esquema XML:

`xmlns: xsi = "http://www.w3.org/2001/XMLSchema-instance"`

Tú puedes usar el atributo **schemaLocation**. Este atributo tiene dos valores, separados por un espacio. El primer valor es **el espacio de nombres a utilizar**. El segundo valor es **la ubicación del esquema XML que se utilizará para ese espacio de nombres**:

`xsi:schemaLocation=https://www.w3schools.com note.xsd`



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



4.5 Modelo de objeto de documento (DOM)

DOM es una abreviatura de Document Object Model. En español podríamos traducirlo por Modelo de Objeto de Documento, a veces referido al DOM con el nombre de jerarquía de objetos del navegador, porque realmente es una estructura jerárquica donde existen varios objetos y unos dependen de otros.

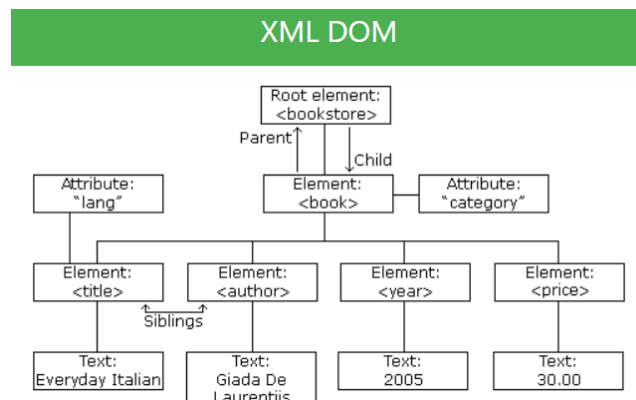
Los objetos del DOM modelizan tanto la ventana del navegador como el historial, el documento o página web, y todos los elementos que pueda tener dentro la propia página, como párrafos, divisiones, tablas, formularios y sus campos, etc. A través del DOM se puede acceder, por medio de Javascript, a cualquiera de estos elementos, es decir a sus correspondientes objetos para alterar sus propiedades o invocar a sus métodos, modificarlos, suprimirlos, crear nuevos elementos y colocarlos en la página, etc.

El DOM está definido y administrado por el W3C, por lo que los distintos navegadores simplemente aplican las especificaciones del World Wide Web Consortium, para dar soporte al DOM en sus aplicaciones. El DOM no sólo permite modificar páginas web en HTML, sino también documentos XML.

A lo largo de la historia de los navegadores, se han ido aplicando en mayor o menor manera las características del DOM. A medida que se sucedían versiones de los navegadores también se iba dando un mayor soporte a las especificaciones del DOM, en lo que se han llamado los niveles del DOM. El primero que tenía disponible por medio de objetos los componentes de la página fue Netscape 2.0, que incorporaba lo que se llama el DOM nivel 0. Actualmente, la última especificación publicada es DOM nivel 4.

El **DOM HTML** define una forma estándar para acceder y manipular documentos HTML. Presenta un documento HTML como una estructura de árbol.

El **DOM XML** define una forma estándar para acceder y manipular documentos XML. Presenta un documento XML como una estructura de árbol.





U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



EL DOM HTML

Todos los elementos pueden ser accedidos a través de DOM HTML, a continuación, verás un ejemplo de esto:

```
<> ejemploDOM.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>:: Ejemplo con DOM ::</title>
7 </head>
8 <body>
9   <h1 id="demo">Este es un encabezado</h1>
10  <p>Este es un párrafo.</p>
11
12  <script>
13    document.getElementById("demo").innerHTML = "Hola chicos!";
14  </script>
15 </body>
16 </html>
```

Browser view: **Hola chicos!**
Este es un párrafo.

Como puedes observar tenemos un simple HTML, con un encabezado y un párrafo, se agrega también una etiqueta `<script>` `</script>`, la cual contiene el método para acceder al elemento que contiene un **id** llamado **demo** y éste a su vez cambia el texto de **“Este es un encabezado”** a **“Hola chicos!”**.

Ahora cambiemos el valor del segundo párrafo de un documento HTML, accederemos a dicho elemento (**p**) a través del método **getElementsByTagName()**:

```
<> ejemploDOM.html > html > body > script
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>:: Ejemplo con DOM ::</title>
7 </head>
8 <body>
9   <h1>Este es un encabezado</h1>
10  <p>Este es un párrafo.</p>
11  <p>Este es en mi segundo párrafo.</p>
12
13  <script>
14    document.getElementsByTagName("p")[1].innerHTML = "El texto de este
15    párrafo fue cambiado."
16  </script>
17 </body>
18 </html>
```

Browser view: **Este es un encabezado**
Este es un párrafo.
El texto de este párrafo fue cambiado.

Es importante que recuerdes que incluso si quisiéramos cambiar el primer párrafo (particularmente en este ejemplo) y solo existiera uno sólo, debemos especificar el índice [0], ya que este método siempre nos retornará un arreglo.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



EL DOM XML

Todos los elementos XML pueden ser accedidos a través del DOM XML.

El DOM XML es un estándar sobre cómo obtener, cambiar, agregar o eliminar elementos XML.

Para obtener el valor de un elemento XML, debes hacerlo a través del siguiente código:

```
txt = xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;
```

Donde xmlDoc – es el documento en sí. Con esto, obtenemos el texto del primer elemento llamado “**título**” en un documento XML. Ver código completo:

The screenshot shows a code editor on the left and a web browser on the right. The code editor displays the following HTML and JavaScript code:

```
<?xml version='1.0' encoding='utf-8'>
<doctype html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>:: Ejemplo DOM XML ::</title>
</head>
<body>
<p>El título del libro consultado a través de DOM XML es:</p>
<p id="dato"></p>
<script>
var text, parse, xmlDoc;

text = "<libreria>"+
"<libro><título>Everyday Italian</título>"+
"<autor>Giada Kundera</autor>"+
"<fecha>2005</fecha></libro>"+
"</libreria>";

parse = new DOMParser();
xmlDoc = parse.parseFromString(text,"text/xml");
document.getElementById("dato").innerHTML = xmlDoc.getElementsByTagName("título")
[0].childNodes[0].nodeValue;
</script>
</body>
</html>
```

The web browser on the right shows the rendered output of the code. The title of the page is "Ejemplo DOM XML ::". The main content of the page is "El título del libro consultado a través de DOM XML es:" followed by "Everyday Italian".

Como podrán observar podemos acceder a los nodos con JavaScript u otros lenguajes de programación.

La interfaz de programación para el DOM se define mediante un conjunto de propiedades y métodos estándar. Las **propiedades** son frecuentemente referenciadas como algo que es (p.e. nodename es “libro”). Algunas de las propiedades típicas de DOM son:

- ✓ **x.nodeName** – El nombre de x
- ✓ **x.nodeValue** – El valor de x
- ✓ **x.parentNode** – El nodo padre de x
- ✓ **x.childNodes** – Los nodos hijos (child) de x
- ✓ **x.attributes** – Los nodos atributos de x



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Los **métodos** son frecuentemente referenciados como algo que está hecho (p.e. delete “book”). Algunos de los **métodos DOM XML** son:

- ✓ **getElementsByTagName(“tagName”)** – Devuelve todos los elementos con un nombre de etiqueta especificado (p.e. `x.getElementsByTagName(“title”)` devuelve todos los elementos “title” debajo del elemento `x`)
- ✓ **x.appendChild(node)** – Inserta un nodo hijo a `x`
- ✓ **x.removeChild(node)** – Quita un nodo hijo de `x`

*Nota: **x** es un objeto de nodo.*

El método `getElementsByTagName()` devuelve una lista de nodos, es decir un arreglo de nodos, lo cual ya mencioné anteriormente y es importante que quede claro.

La propiedad de longitud define la longitud de una lista de nodos. Puedes recorrer dicha lista usando la propiedad de longitud de la siguiente forma:

```
var x = xmlDoc.getElementsByTagName("title");

for (i = 0; i < x.length; i++) {
    // do something for each node
}
```

En el DOM XML, cada nodo es un objeto y los objetos tienen métodos y propiedades, a los que JavaScript puede acceder y manipular:

- ✓ La propiedad **documentElement** del documento XML es un nodo raíz (root).
- ✓ La propiedad **nodeName** de un nodo especifica el nombre del nodo (es de solo lectura, `nodeName` de un elemento nodo es el mismo como el nombre de la etiqueta, `nodeName` de un nodo atributo es el nombre del atributo, `nodeName` de un nodo texto, es siempre el `#texto` y `nodeName` del nodo del documento siempre es el `#documento`)
- ✓ La propiedad **nodeValue** especifica el valor de un nodo (`nodeValue` para nodos de elementos es indefinida, `nodeValue` para nodos de texto es el propio texto, `nodeValue` para nodos atributo es el valor del atributo)
- ✓ La propiedad **nodeType** de un nodo especifica el tipo del nodo (es de solo lectura).

Los tipos de nodos más importantes son:

- Elemento (1)
- Atributo (2)
- Text (3)
- Comentario (8)
- Documento (9)



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



En los siguientes ejemplos podrán observar cómo se utilizan algunas las propiedades antes enlistadas:

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        myFunction(this);
    }
};
xhttp.open("GET", "books.xml", true);
xhttp.send();

function myFunction(xml) {
    var xmlDoc = xml.responseXML;
    document.getElementById("demo").innerHTML =
        xmlDoc.documentElement.nodeName;
}
</script>

</body>
</html>
```

bookstore

nodeName

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        myFunction(this);
    }
};
xhttp.open("GET", "books.xml", true);
xhttp.send();

function myFunction(xml) {
    var xmlDoc = xml.responseXML;
    var x = xmlDoc.getElementsByTagName("title")[0].childNodes[0];
    document.getElementById("demo").innerHTML = x.nodeValue;
}
</script>

</body>
</html>
```

Everyday Italian

nodeValue



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



```
<!DOCTYPE html>
<html>
<body>

<p id="demo1"></p>
<p id="demo2"></p>

<script>
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        myFunction(this);
    }
};
xhttp.open("GET", "books.xml", true);
xhttp.send();

function myFunction(xml) {
    var xmlDoc = xml.responseXML;
    var x;
    x = xmlDoc.getElementsByTagName("title")[0].childNodes[0];
    document.getElementById("demo1").innerHTML = x.nodeValue;
    x.nodeValue = "Easy Cooking";
    x = xmlDoc.getElementsByTagName("title")[0].childNodes[0];
    document.getElementById("demo2").innerHTML = x.nodeValue;
}
</script>
</body>
</html>
```

Everyday Italian

Easy Cooking

Cambiar el texto en un nodo

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        myFunction(this);
    }
};
xhttp.open("GET", "books.xml", true);
xhttp.send();

function myFunction(xml) {
    var xmlDoc = xml.responseXML;
    document.getElementById("demo").innerHTML =
        xmlDoc.documentElement.nodeName + "<br>" +
        xmlDoc.documentElement.nodeType;
}
</script>

</body>
</html>
```

bookstore
1

Obtenemos el nombre y tipo nodo de un elemento nodo



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

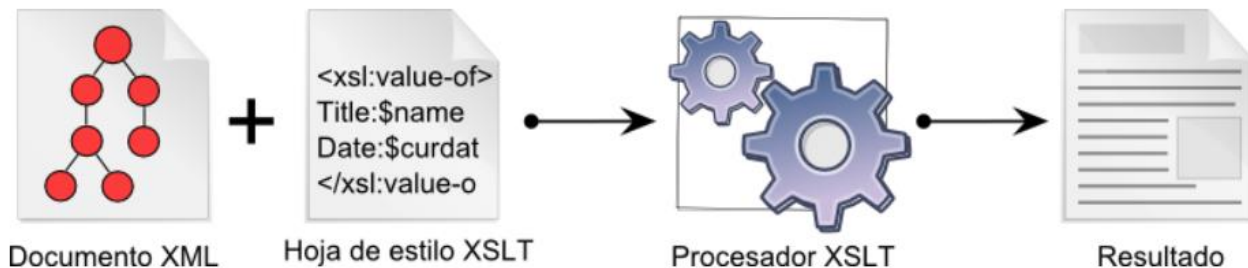
Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



4.6 XSLT

XSL (eXtensible Stylesheet Language) es un lenguaje de estilo para XML.

XSLT significa Transformaciones XSL. Es un lenguaje para transformar documentos XML, como se ilustra en la siguiente imagen:



- El documento XML es el documento inicial a partir del cual se va a generar el resultado.
- La hoja de estilo XSLT es el documento que contiene el código fuente del programa, es decir, las reglas de transformación que se van a aplicar al documento inicial.
- El procesador XSLT es el programa de ordenador que aplica al documento inicial las reglas de transformación incluidas en la hoja de estilo XSLT y genera el documento final.
- El resultado de la ejecución del programa es un nuevo documento (que puede ser un documento XML o no).

XSLT se utiliza para obtener a partir de un documento XML otros documentos (XML o no). A un documento XML se le pueden aplicar distintas hojas de estilo XSLT para obtener distintos resultados y una misma hoja de estilo XSLT se puede aplicar a distintos documentos XML.

El lenguaje XSLT está normalizado por el W3C, que ha publicado tres versiones de este lenguaje:

- noviembre de 1999: [XSLT 1.0](#)
- enero de 2007: [XSLT 2.0](#)
- junio de 2017: [XSLT 3.0](#)

En otras palabras, **XSLT se puede entender como las hojas de estilo para el XML.**



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Las XSLT no se escriben como una secuencia de instrucciones, sino como una colección de plantillas (template rules). Cada plantilla establece cómo se transforma un determinado elemento definido mediante expresiones XPath (que es una cadena de texto que representa un recorrido en el árbol del documento, el resultado que se obtiene son todos los nodos que se ajustan a la expresión). Dicha transformación se realiza de la siguiente manera:

- El procesador analiza el documento y construye el árbol del documento.
- El procesador recorre el árbol del documento desde el nodo raíz.
- En cada nodo recorrido, el procesador aplica o no alguna plantilla:
 - Si a un nodo no se le puede aplicar ninguna plantilla, su contenido se incluye en el documento final (el texto del nodo, no el de los nodos descendientes). A continuación, el procesador recorre sus nodos hijos.
 - Si a un nodo se le puede aplicar una plantilla, se aplica la plantilla. La plantilla puede generar texto que se incluye en el documento final. En principio, el procesador no recorre sus nodos hijos, salvo que la plantilla indique al procesador que sí que deben recorrerse los nodos hijos.
- Cuando el procesador ha recorrido el árbol, se ha terminado la transformación.

Una hoja de estilo XSLT es un documento XML que contiene al menos las siguientes etiquetas:

```
pruebaEstilo.xml X
pruebaEstilo.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
   version="1.0">
3
4
5  </xsl:stylesheet>
```

- La declaración xml `<?xml>`, propia de cualquier documento XML.
- La instrucción `<xsl:stylesheet>` es la etiqueta raíz de la hoja de estilo, sus atributos indican la versión y el espacio de nombres correspondiente.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Dentro de la instrucción `<xsl:stylesheet>` se pueden encontrar los llamados elementos de alto nivel y las plantillas, como en el siguiente ejemplo:

```
pruebaEstilo.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
3
4      <xsl:output method="xml" />
5
6      <xsl:template match="/">
7          </xsl:template>
8
9  </xsl:stylesheet>
10
```

Estas etiquetas son:

- El elemento de alto nivel `<xsl:output>` indica el tipo de salida producida.
- La instrucción `<xsl:template>` es una plantilla.
 - El atributo `match` indica los elementos afectados por la plantilla y contiene una expresión XPath.
 - El contenido de la instrucción define la transformación a aplicar (si la instrucción no contiene nada, como en el ejemplo anterior, sustituirá el nodo por nada, es decir, eliminará el nodo, aunque conservará el texto contenido en el elemento).

Cuando se aplica una plantilla a un nodo, en principio no se recorren los nodos descendientes. Para indicar que sí queremos recorrer los nodos descendientes y aplicarles las plantillas que les correspondan, hay que utilizar la instrucción `<xsl:apply-templates />`, como en el ejemplo siguiente:

```
pruebaEstilo.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
3
4      <xsl:output method="xml" />
5
6  <xsl:template match="/">
7      <xsl:apply-templates />
8  </xsl:template>
9
10 <xsl:template match="elemento">
11 </xsl:template>
12
13 </xsl:stylesheet>
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Marcado Extensible (XML)



Para enlazar documentos XML con **hojas de estilo XSLT**, debemos utilizar la instrucción de procesamiento `<?xml-stylesheet ?>`, la misma que permite asociar hojas de estilo CSS, ésta va al principio del documento, después de la declaración XML:

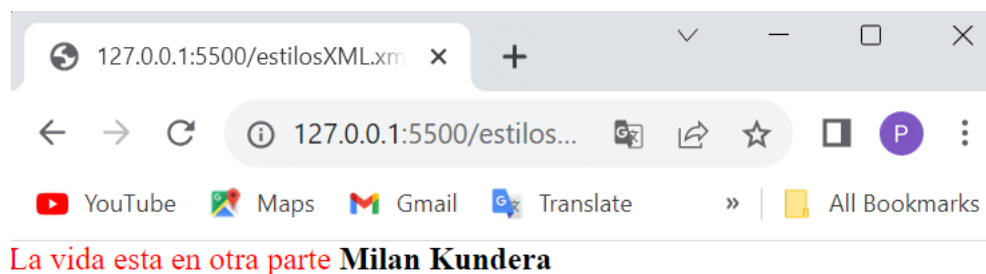
```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="ejemplo.xsl"?>
```

Cuando se visualiza en un navegador web un documento XML enlazado con una hoja de estilo XSLT, los navegadores muestran el resultado de la transformación, aunque si se muestra el código fuente de la página, los navegadores muestran el documento XML original.

Ahora bien, para comprender mejor como se utilizan las XSLT, comenzaremos por ver un ejemplo de un XML aplicándole estilos desde una CSS:

```
estilosXML.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?xml-stylesheet href="estilos/pruebaEstilo.css" ?>
3
4 <libro>
5   <titulo>La vida esta en otra parte</titulo>
6   <autor>Milan Kundera</autor>
7   <fechaPublicacion año="1973" />
8 </libro>
```

```
estilos > # pruebaEstilo.css > autor
1 titulo{color: red;}
2 autor{font-weight: bold;}
```



En este ejemplo podemos observar que la información es desplegada de corrido (en una sola línea), entonces en nuestra hoja de estilo para el documento XML hay que definir propiedades que normalmente no se suelen definir, por ejemplo: la propiedad “display”, que es una de las más importantes y establece el modo de visualización del documento.



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)

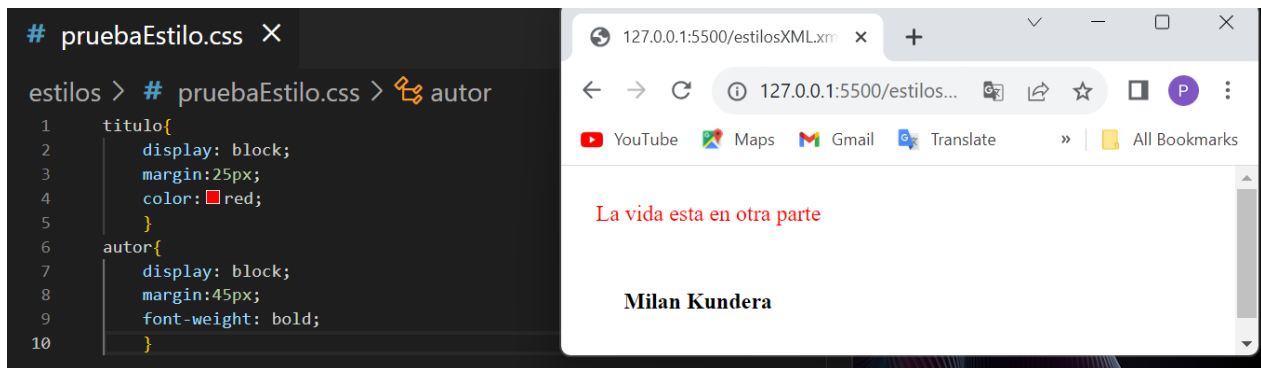


Los modos de visualización más utilizados son:

- **none** (ninguno) permite ocultar elementos.
- **block** (bloque) es el modo de visualización de los elementos de tipo bloque, como párrafos (`<p>`), encabezados (`<h1>`, `<h2>`, ...), etc. El elemento ocupa toda la ventana en horizontal y ocupa el espacio vertical necesario para alojar el contenido del elemento.
- **inline** (elemento en línea) es el modo de visualización de etiquetas como `<strong>`, `<em>`, `<span>`, `<a>`, etc. El elemento sólo ocupa el espacio necesario para alojar el contenido del elemento. En HTML los elementos en-línea tienen que estar contenidos dentro de elementos de bloque.
- **list-item** (lista) es el modo de visualización de elementos de listas como `<li>`, `<dd>`, `<dt>`.
- **table** (tabla), **table-row** (fila) y **table-cell** (celda) se combinan para mostrar elementos en forma de tabla.

Recuerden que éstos ya los aprendimos y trabajamos en HTML.

Aquí les muestro el mismo ejemplo aplicando algunas propiedades:



Retomando el uso de las plantillas (“templates”) en los XSLT, que vimos más arriba, dijimos que, si una plantilla está vacía y se aplica al nodo raíz, el procesador no genera ningún resultado en el documento final ni recorre ningún nodo hijo.

Por lo que, revisaremos la instrucción `<xsl:value-of>`, la cual extrae el contenido del nodo seleccionado.

En el siguiente ejemplo, el documento final contiene los autores de los libros porque la plantilla los genera con la instrucción `<xsl:value-of>`. La plantilla se aplicará al nodo `<libro>`, sus hijos (`<titulo>`, `<autor>` y `<fechaPublicacion>`) no se recorren. Por eso, los títulos de los libros no aparecen en el documento final.

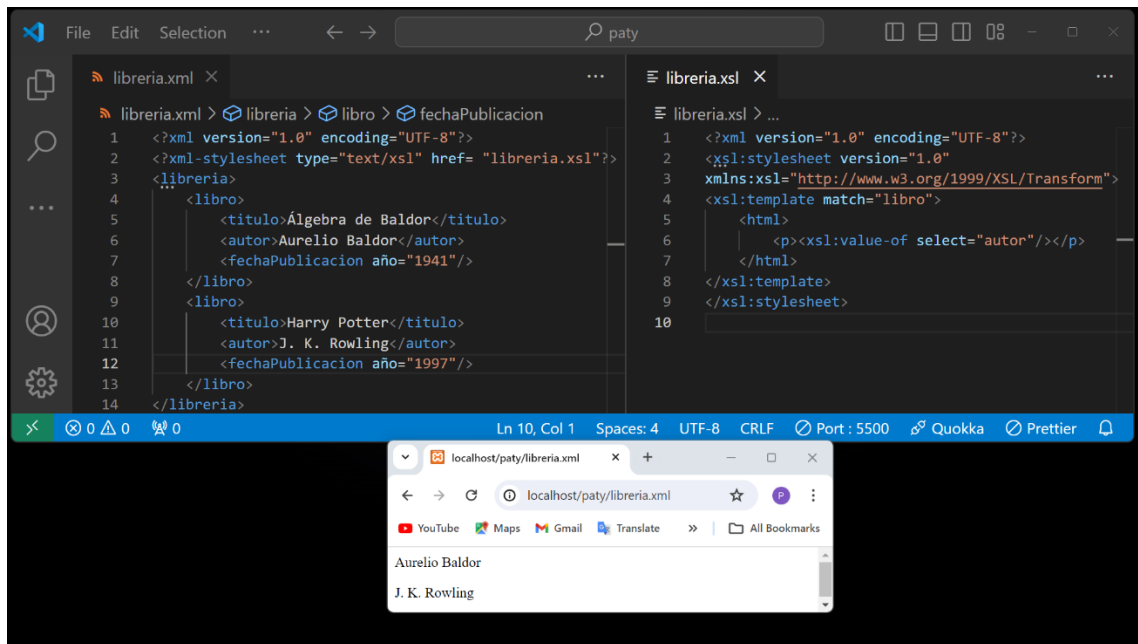


U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Nota: Google Chrome no muestra los documentos XML que enlazan a hojas de estilo XSLT abiertos como archivos locales (file:///...).



Como podemos observar, se obtienen los autores de los libros, con un salto de línea debido al formato que el elemento `<p>` tiene por default.

En la propiedad **select**, además del nombre del elemento podemos utilizar: “.” que extrae el contenido del nodo (el punto hace referencia al propio elemento).

También podemos extraer los valores de los atributos, utilizando **@**.

Por otro lado, podemos aplicar la instrucción `<xsl:apply-templates>` la cual, hace que se apliquen a los subelementos las reglas que les sean aplicables.

<pre><?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template match="/"> <html> <h1>Autores</h1> <xsl:apply-templates /> </html> </xsl:template> <xsl:template match="libro"> <p><xsl:value-of select="autor"/></p> </xsl:template> </xsl:stylesheet></pre>	<pre><?xml version="1.0" encoding="UTF-8"?> <html><h1>Autores</h1> <p>Milan Kundera</p> <p>Mario Vargas Llosa</p> <p>Mario Vargas Llosa</p> </html></pre>
---	---



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



Al transformar un documento, los procesadores XSLT incorporan saltos de línea y espacios en blanco en el resultado, pero no lo hacen de forma uniforme. Por ejemplo, XML Copy Editor y Notepad++ (con el plug-in XML Tools) producen diferentes resultados.

No parece haber una solución sencilla que funcione en todos los procesadores, pero sí soluciones que funcionen en cada uno de ellos.

La instrucción `<xsl:strip-space>` en el caso de XML Copy Editor y `<xsl:text>` para Notepad++ con XML Tools.

La instrucción `<xsl:attribute>` permite generar un atributo y su valor. Se utiliza cuando el valor del atributo se obtiene a su vez de algún nodo.

Por ejemplo, a partir del siguiente documento XML, se quiere generar la etiqueta `<img>`, en la que el valor del atributo `src` sea el contenido de la etiqueta `<imagen>`.

```
<?xml version="1.0" encoding="UTF-8"?>
<licencias>
  <licencia>
    <nombre>Creative Commons By - Share Alike</nombre>
    <imagen>cc-bysa-88x31.png</imagen>
  </licencia>
</licencias>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  version="1.0">
  <xsl:template match="/">
    <html>
      <xsl:apply-templates />
    </html>
  </xsl:template>

  <xsl:template match="licencia">
    <p><img>
      <xsl:attribute name="src">
        <xsl:value-of select="imagen" />
      </xsl:attribute>
    </img>
  </p>
  </xsl:template>
</xsl:stylesheet>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<html>
  <p></p>
</html>
```

También podemos trabajar con los siguientes elementos:

`<xsl:for-each>` - permite hacer bucles en XSLT

```
<xsl:for-each select="catalog/cd">
  <tr>
    <td><xsl:value-of select="title"/></td>
    <td><xsl:value-of select="artist"/></td>
  </tr>
</xsl:for-each>
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Marcado Extensible (XML)



<xsl:sort> - es usado para ordenar la salida

```
<xsl:for-each select="catalog/cd">
  <xsl:sort select="artist"/>
  <tr>
    <td><xsl:value-of select="title"/></td>
    <td><xsl:value-of select="artist"/></td>
  </tr>
</xsl:for-each>
```

<xsl:if> - es usado para poner una condición contra el contenido del archivo XML

```
<xsl:if test="expression">
  ...some output if the expression is true...
</xsl:if>
```

<xsl:choose> - se utiliza junto con **<xsl:when>** y **<xsl:else>** para expresar múltiples pruebas condicionales.

```
<xsl:choose>
  <xsl:when test="expression">
    ... some output ...
  </xsl:when>
  <xsl:otherwise>
    ... some output ....
  </xsl:otherwise>
</xsl:choose>
```



U.A. Tecnologías para la Desarrollo de Aplicaciones Web

Unidad Temática IV. Lenguaje de Mercado Extensible (XML)



4.7 Familia de dialectos XML

Dentro de los dialectos XML tenemos:

- ✓ [SVG](#) Imágenes vectoriales
- ✓ [X3D Imágenes 3D](#)
- ✓ [GML](#) Representación geográfica
- ✓ [WDDX](#)
- ✓ [XHTML](#) HTML extensible (reformulación del [HTML](#) en [XML](#))
- ✓ [MathML](#) Ecuaciones matemáticas
- ✓ [XBRL](#) Información financiera
- ✓ [RDF/XML](#) (formulación en XML de [RDF](#))
- ✓ XTM
- ✓ [EBML](#) (Xml Binario)
- ✓ [RSS](#) Sindicación de noticias
- ✓ lexml Aplicación de XML al ámbito legal.

Una alternativa a los dialectos XML son los *lenguajes de marcas ligeros*, que en determinadas aplicaciones pueden ser más apropiados.